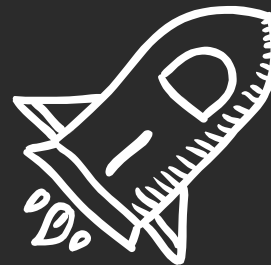




Semana 11

¡Bienvenidos!



Temas de hoy

1 Modelos y Bases de
datos en Django

3 Migraciones

2 Definición de modelos

4 Administración de Django

1

Modelos y Bases de datos en Django

¿Te acordás que con Python, utilizando la librería correspondiente, podíamos crear y utilizar una base de datos en su plenitud?

Gracias a esto, nuestras aplicaciones en el proyecto de Django van a estar conectadas a una base de datos en un futuro, y este framework, Django, nos provee una solución con los modelos



Modelos en Django

¿Qué es un modelo?

Los modelos en Django, son las representaciones en Python de una tabla en la base de datos.

Es la forma en la que Django puede interactuar con los datos que deseamos almacenar y manipular.

Cada **modelo** que definamos va a corresponder a **una tabla** en particular, y cada **atributo** perteneciente a un modelo representa **una columna** en esa tabla.

Por otro lado, cada aplicación de nuestro proyecto tendrá su propio archivo de modelos, por lo tanto la suma de **todos** los archivos *models* conforman la estructura **total** de nuestra base de datos.

La definición de los modelos en Django es independiente de la base de datos que se utiliza.

Los modelos, como bien dijimos, están definidos normalmente en el archivo **models.py** de una aplicación y se los implementa como una clase hija o subclase de *django.db.models.Model*, y puede incluir campos, métodos y metadata.

Con Django, en lugar de escribir las consultas SQL directamente para manipular la base de datos, utiliza un sistema llamado **ORM (Object-Relational Mapper)** que nos permite interactuar con la base de datos utilizando Python como lenguaje haciendo así el proceso mucho más intuitivo y reduciendo la posibilidad a errores.

Cuando hablamos de definir un modelo en Django nos referimos a un proceso más allá de simplemente enumerar campos, si no que buscamos **conceptualizar** cómo vamos a relacionar los datos en nuestra aplicación y cómo queremos que interactúen entre sí.

2

Definición de modelos

Cuando hablamos de definir un modelo, hablamos de una solución al almacenamiento de datos, ya que, sin importar la base de datos que usemos, con nuestros modelos definidos, Django se va a encargar del resto... mejor, lo vemos con un ejemplo 🤔

Definición de modelos

Ejemplo

Por ejemplo, en el caso de una biblioteca tendremos un modelo para los libros

```
from django.db import models

class Libro(models.Model):
    titulo = models.CharField(max_length=200)
    autor = models.CharField(max_length=100)
    fecha_publicacion = models.DateField()
    isbn = models.CharField(max_length=13, unique=True)

    def __str__(self):
        return self.titulo
```

Para la definición de nuestros modelos requerimos importar “**models**” de **django.db** como lo hacemos en la primera línea.

Luego nuestro modelo **Libro** (declarada como una clase) debe heredar de la clase **Model** perteneciente a la librería importada (models).

Una vez que la herencia está definida, podemos empezar a declarar nuestros atributos y definir los tipos de datos que almacenarán utilizando **models.tipo_de_dato(propiedad)**.

Si bien, es un modelo de Django, no pierde las características de una clase de Python, por lo tanto tenemos la posibilidad de utilizar dunder methods en ella como lo es `__str__()`.

Para investigar **todos** los tipos de datos que nos provee la librería *models.Model* debemos acceder a su documentación 🤖

▶ [Documentación oficial](#)

▶ [Resumen general en GeeksforGeeks](#)

Definición de modelos

Además de definir los atributos de nuestra clase con sus tipos de datos, tenemos la posibilidad de declarar las **relaciones** entre modelos.

Por ejemplo, ¿qué pasa si un libro puede tener múltiples autores?

Eso significa que tendremos una relación de **muchos a muchos (m:n)** donde un autor puede escribir muchos libros, y un libro puede ser escrito por muchos autores.

Ante una situación como esta, Django nos permite utilizar **models.ManyToManyField(Modelo)** como lo podemos ver en el atributo “*autores*” del siguiente ejemplo, donde pasamos como argumento del método al modelo que hará referencia en ese campo.

En este caso, el atributo “*autores*” del modelo **Libro** podrá guardar uno o más registros del modelo “*Autor*”.

```
from django.db import models

class Autor(models.Model):
    nombre = models.CharField(max_length=100)
    nacionalidad = models.CharField(max_length=50)

    def __str__(self):
        return self.nombre

class Libro(models.Model):
    titulo = models.CharField(max_length=200)
    autores = models.ManyToManyField(Autor)
    fecha_publicacion = models.DateField()
    isbn = models.CharField(max_length=13, unique=True)

    def __str__(self):
        return self.titulo
```

Al igual que vimos en el pasaje de MER a tablas como resolver este tipo de relaciones, al momento de declarar un atributo con la relación *ManyToManyField()*, Django **internamente** crea una tabla intermedia en la base de datos para relacionar ambos modelos correctamente.



Entonces...

... la definición de modelos es una facilidad que nos provee Django para una comunicación más intuitiva e independiente del motor de la base de datos. Además de proporcionarnos mayor facilidad al momento de crear las tablas y posteriormente de poder manipularlas dentro de nuestro proyecto.

3

Migraciones

De momento tenemos creado todos nuestros modelos acorde a nuestro ejemplo, pero ahora nos falta un paso esencial y es que las nuevas implementaciones impacten en la base de datos.



Veamos cómo hacerlo.

Migraciones: Definición y características

Dando marcha a la BD

Una vez **creados** o **modificados** nuestros modelos, necesitamos reflejar estos cambios en nuestra base de datos utilizando las **migraciones**.

Las migraciones son archivos que registran los **cambios** en los modelos y Django los usa para **sincronizar** la estructura entre nuestra base de datos y los modelos.

Podemos interpretarlos como las actualizaciones de las aplicaciones de tu celular. Cada vez que hay una nueva versión, se instala un parche que actualiza la aplicación sin perder los datos ni configuraciones que tenías anteriormente.

Las migraciones logran el mismo efecto permitiéndonos evolucionar la estructura de nuestra base de datos sin tener que reconstruirla desde cero cada vez que necesitamos realizar un cambio en ella.

Importancia de las migraciones:

Las migraciones nos permiten...

- **...versionar los cambios:** Las migraciones nos dejan un registro de cómo fue cambiando y evolucionando nuestra estructura de datos a lo largo del tiempo.
- **...mejorar la colaboración en equipo:** En proyectos con varios desarrolladores, las migraciones nos aseguran que todos trabajan con la misma estructura de base de datos.
- **...desplegar de manera más fácil:** Es recurrente mover la aplicación entre desarrollo, pruebas, producción, etc., y las migraciones nos facilitan la instalación y actualización de la base de datos.

Migraciones: Ejemplo

```
from django.db import models

class Libro(models.Model):
    titulo = models.CharField(max_length=200)
    autor = models.CharField(max_length=100)
    fecha_publicacion = models.DateField()
    isbn = models.CharField(max_length=13, unique=True)
    categoria = models.CharField(max_length=55)

    def __str__(self):
        return self.titulo
```

En el ejemplo podemos ver que agregamos un nuevo campo “categoria” que no se encontraba en la estructura inicial.

Al nosotros realizar un cambio en la estructura, debemos ejecutar en la terminal el comando “**python manage.py makemigrations**” para que analice y detecte los cambios que sufrieron los archivos models. Si encuentra alguna diferencia, creará una **nueva migración**.

```
$ python manage.py makemigrations
Migrations for 'noticias':
  apps\noticias\migrations\0001_initial.py
  + Create model Libro
```

Al crear esta migración, deberemos aplicarla a la base de datos utilizando “**python manage.py migrate**”.

```
$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, noticias, sessions
Running migrations:
  Applying noticias.0001_initial... OK
```

4

Administración de Django

Django nos prepara de antemano una interfaz de administración para que podamos gestionar nuestros modelos en funcionamiento de manera intuitiva e interactiva.

Veamos como implementar 🙌

Administración de Django

Una solución rápida

El **sitio de administración de Django** es una herramienta que nos provee este Framework para poder **gestionar** nuestros modelos y datos de manera más sencilla y eficiente.

Una vez que tenemos definidos nuestros modelos en nuestra aplicación y habiendo hecho las migraciones correspondientes, Django nos genera automáticamente una interfaz para poder administrarlo.

→ Seguridad y permisos

Este sitio de administración está protegido por autenticación y permisos preestablecidos, es decir, que solo usuarios autorizados pueden acceder y realizar acciones, lo que nos garantiza seguridad en nuestros datos.

Además tenemos la posibilidad de definir diferentes niveles de acceso o roles para distintos usuarios, permitiéndonos controlar quién puede agregar, modificar o eliminar registros.

→ Extensibilidad:

Por otro lado, el administrador es altamente extensible debido a que tenemos la posibilidad de integrar plugins, personalizar la apariencia y agregar nuevas funcionalidades avanzadas. Esto nos ahorra tiempo y recursos al no tener que desarrollar desde cero un panel de administración.

Administración de Django: Superuser

Al ingresar al **sitio de administración de Django** (<http://localhost:8000/admin>) veremos que nos piden credenciales de acceso:



Estas credenciales las debemos de generar creando un **superuser** con permiso absoluto sin ninguna restricción. Para ello ocuparemos en la terminal el comando:
python manage.py createsuperuser

```
$ python manage.py createsuperuser
Nombre de usuario (leave blank to use 'admin'): Info
Dirección de email: infor@matorio.com
Password:
Password (again):
Superuser created successfully.
```

Y completamos los campos con las nuevas credenciales de acceso.

Una vez creado el superuser, tendremos acceso a nuestro administrador de Django.



Administración de Django: Registro de modelos

Al crear una nueva aplicación de Django en nuestro proyecto, nos genera un archivo llamado **admin.py**.

Este cumple la función de configurar cómo se van a mostrar y gestionar los modelos dentro de nuestro sitio de administración.

Como podemos notar en el administrador de Django, solo vemos los modelos de **Grupos** y **Usuarios** que nos provee Django de forma predeterminada pero aún habiendo migrado el modelo de **Libro**, no podemos visualizar lo, ni interactuar con él.

Es por ello que debemos configurar el archivo admin.py para que implementar nuestro modelo en el administrador:

```
from django.contrib import admin
from .models import Libro

admin.site.register(Libro)
```

Importamos el modelo en este archivo y lo pasamos como argumento al momento de registrarlo (*admin.site.register()*). Dandonos como resultado:

Administración de sitio

AUTENTICACIÓN Y AUTORIZACIÓN

Grupos

Usuarios

Administración de sitio

AUTENTICACIÓN Y AUTORIZACIÓN

Grupos

Usuarios

NOTICIAS

Libros

Agregar libro

Título:

Autor:

Fecha publicación:

libro:

Categoría:

GUARDAR

Guardar y agregar otro

Guardar y continuar editando

Administración de Django: Personalización de la vista

Siguiendo con el ejemplo anterior, si ingresamos a nuestro nuevo modelo implementado e intentamos agregar un nuevo registro desde nuestro administrador, nos mostrará un formulario con los campos que conforman nuestro modelo listo para ser usado:



Pero el archivo admin.py también nos permite personalizar cómo queremos disponer la información de los modelos en el sitio de administración. Por ejemplo, podemos elegir que campos mostrar, agregar filtros o funcionalidades adicionales:

```
from django.contrib import admin
from .models import Libro

class LibroAdmin(admin.ModelAdmin):
    fields = ('titulo', 'autor', 'isbn', )
    list_display = ('titulo', 'isbn')
    search_fields = ('titulo',)
    list_filter = ('isbn',)

admin.site.register(Libro, LibroAdmin)
```

Vamos a profundizar en estos atributos pertenecientes al **ModelAdmin** 🙌

Administración de Django: Personalización de la vista

```
fields = ('titulo', 'autor', 'isbn', )
```

El atributo **fields** nos permite seleccionar qué campos se mostrarán al momento de agregar un nuevo registro:

Agregar libro

Título:

Autor:

Isbn:

```
list_display = ('titulo', 'isbn')
```

Con el atributo **list_display** definiremos los campos que vamos a poder visualizar cuando ingresamos a este modelo en el administrador:

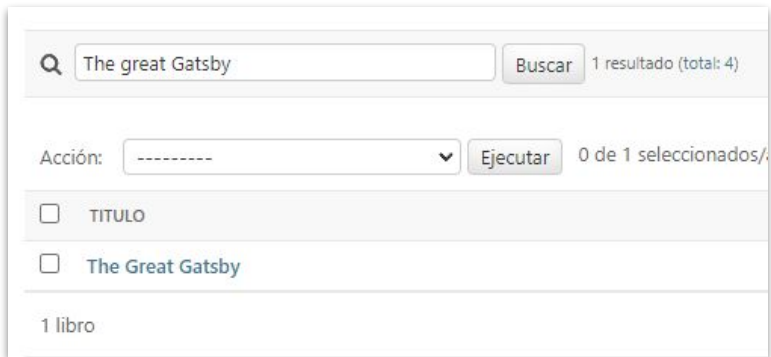


☐ TITULO	ISBN
☐ Pride and Prejudice	9781503290563
☐ The Great Gatsby	9780743273565
☐ 1984	9780451524935
☐ To Kill a Mockingbird	9780061120084
4 libros	

Administración de Django: Personalización de la vista

```
search_fields = ('titulo',)
```

El atributo **search_fields** define cuál será el campo por el que haremos las búsquedas en nuestro administrador:



Acción:	-----	Ejecutar	0 de 1 seleccionados/
☐	TITULO		
☐	The Great Gatsby		
1 libro			

```
list_filter = ('isbn',)
```

El atributo **list_filter** nos proporciona un cuadro filtrado por el campo a elección:



FILTRAR
☒ Mostrar recuentos
1 Por isbn
Todos/as
9780061120084
9780451524935
9780743273565
9781503290563



Entonces...

... el administrador que nos provee Django nos permite tener una herramienta similar pero más acotada a los gestores de base de datos que vimos en clases anteriores pero sin el uso de SQL. Nos permite tener un acceso sencillo, interactivo y eficiente a nuestros modelos y a los datos que almacene solamente configurando un archivo.



Lo que vimos hoy 🤔

La interacción de nuestro proyecto con la base de datos cada vez se entrelaza más gracias a las posibilidades que Django nos ofrece. Podemos crear la estructura de nuestra base de datos mediante la creación de nuestros modelos, impactar dichos cambios independientemente del motor que utilizamos gracias a las migraciones y a crear, modificar o eliminar registros en nuestro nuevo panel de administración.



**¡Nos vemos
En la próxima
clase!**

